



Algoritmos y estructuras de datos.

Trabajo práctico N°1.

Informe Proyecto 1- TAD Lista Doblemente Enlazada (ListaDobleEnlazada)

Integrantes del grupo: - Hasenauer, Fernanda.
-Hernandez, Tomas
-Leguizamon, Augusto.

Fecha de entrega: 28/9/26.

Link del repositorio:

[AyED2026c2-Hernandez-Hasenauer-Leguizamon/TrabajoPractico_1/proyecto_1 at main · Hernandez-TV/AyED2026c2-Hernandez-Hasenauer-Leguizamon](#)

Introduccion

El objetivo de este trabajo práctico fue implementar un TAD Lista Doblemente Enlazada que permita almacenar elementos de cualquier tipo comparable, respetando las operaciones básicas de inserción, extracción, copia, inversión y concatenación. Esta estructura fue elegida porque permite eficiencia en la inserción y extracción de elementos en los extremos de la lista, así como recorrerla en ambos sentidos mediante punteros a nodos anteriores y siguientes.

Estructura de la lista:

La lista se compone de nodos, cada uno con:

- Dato: elemento almacenado (`_dato`).
- Siguiente: referencia al siguiente nodo (`_siguiente`).
- Anterior: referencia al nodo anterior (`_anterior`).

La lista mantiene punteros a:

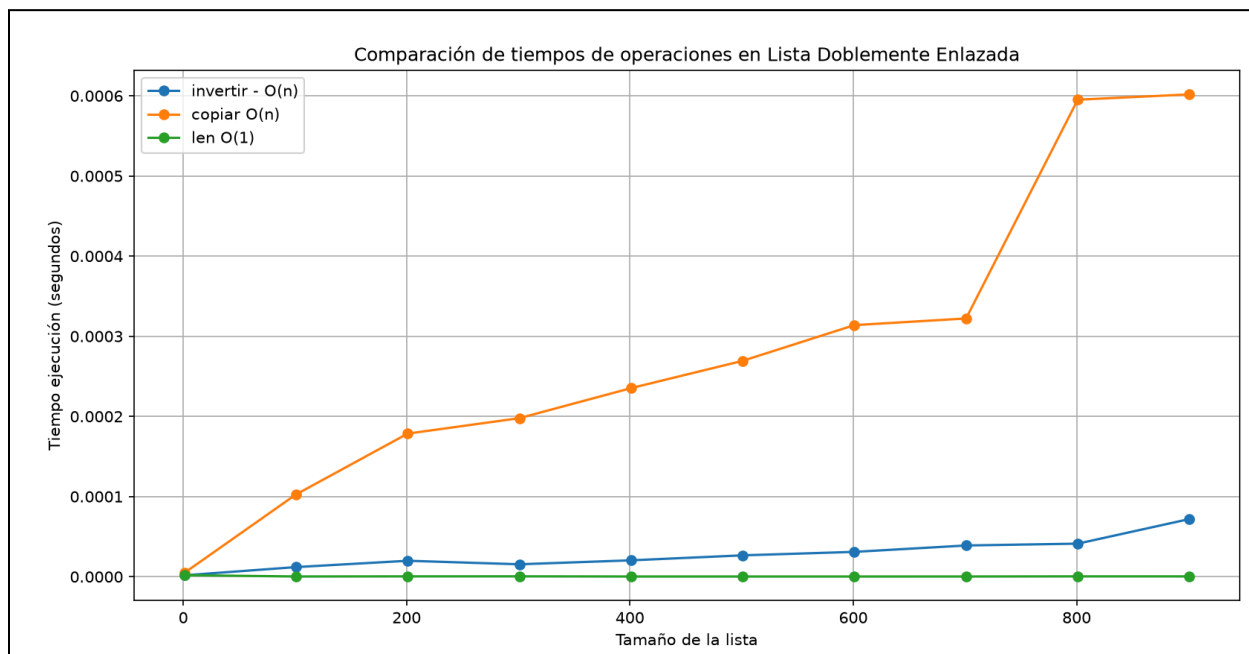
- Cabeza: referencia al primer nodo (`_cabeza`).
- Cola: referencia al último nodo (`_cola`).
- Tamaño: número actual de elementos (`_tamaño`).

Métodos implementados:

Método	Descripción	Complejidad
<code>esta_vacia()</code>	Devuelve True si la lista está vacía	$O(1)$
<code>agregar_al_inicio(ítem)</code>	Inserta un elemento al principio de la lista	$O(1)$
<code>agregar_al_final(ítem)</code>	Inserta un elemento al final de la lista	$O(1)$
<code>insertar(ítem, pos)</code>	Inserta en la posición indicada; si no se indica, al final	$O(n)$ en peor caso
<code>extraer(pos)</code>	Elimina y devuelve el elemento en la posición; si no, el último	$O(1)$ extremos / $O(n)$ general
<code>copiar()</code>	Devuelve una copia independiente de la lista	$O(n)$
<code>invertir()</code>	Invierte el orden de los elementos in-place	$O(n)$
<code>concatenar(lista)</code>	Concatena otra lista al final	$O(n)$
<code>__len__()</code>	Devuelve el número de elementos de la lista	$O(1)$
<code>__add__(lista)</code>	Retorna una nueva lista con los elementos de ambas	$O(n)$
<code>__iter__()</code>	Permite iterar con un ciclo for mediante un generador	$O(n)$

Todos los métodos fueron implementados sin usar estructuras auxiliares innecesarias (como listas nativas de Python), cumpliendo con la eficiencia en memoria requerida por la consigna.

Análisis de Resultados Empíricos y Complejidad:



- Método `len()` — Complejidad $O(1)$: Muestra una curva totalmente horizontal pegada a 0.0 segundos. Esto confirma que la consulta del tamaño es de tiempo constante, ya que accede directamente a la variable interna de la instancia (`_tamanio`) en lugar de recorrer los nodos.
- Método `invertir()` — Complejidad $O(n)$: Presenta una línea ascendente con comportamiento estrictamente lineal. La función invierte las referencias siguiente y anterior de cada nodo in-place en una sola pasada, alcanzando la máxima eficiencia espacial $O(1)$ auxiliar.
- Método `copiar()` — Complejidad $O(n)$: Exhibe una pendiente lineal similar a `invertir()`, pero con una constante de tiempo superior. Esto se debe a que, además de recorrer los N nodos, asigna dinámicamente nuevos objetos `Nodo` en memoria por cada elemento copiado.

Conclusión:

1. Validación Empírica del Análisis a Priori: Las mediciones de tiempo de ejecución realizadas con `time.perf_counter()` confirman de manera precisa los órdenes de complejidad asintótica deducidos teóricamente. El método `len()` exhibe una curva estrictamente constante $O(1)$ al acceder de forma directa al atributo privado de instancia `_tamanio`, evitando cualquier recorrido sobre los nodos de la estructura.

2. Eficiencia Temporal, Espacial y Análisis de Pendientes: Las operaciones `invertir()` y `copiar()` demuestran un crecimiento lineal $O(n)$ proporcional a N . Destaca la implementación de `invertir()`

por lograr la máxima eficiencia espacial $O(1)$ auxiliar mediante el reordenamiento in-place de las referencias siguiente y anterior sin instanciar nuevos objetos. Por su parte, la mayor pendiente observada en la curva de *copiar()* refleja la sobrecarga constante por asignación de memoria dinámica al instanciar objetos Nodo en el Heap, conservando la misma clase de complejidad lineal $O(n)$.

3. Adherencia a Restricciones y Estándares de la Cátedra: La solución cumple estrictamente con la restricción de no emplear listas nativas de Python ni estructuras auxiliares innecesarias para la manipulación interna. La gestión de encapsulamiento mediante decoradores `@property`, el manejo adecuado de excepciones (`IndexError`, `TypeError`) y la superación exitosa del 100% de los tests unitarios avalan la solidez y robustez de la arquitectura implementada.

Declaración de Uso Ético de Inteligencia Artificial Herramientas utilizadas:

Para el desarrollo de este proyecto se utilizó la herramienta de Inteligencia Artificial Gemini Notebook como asistente interactivo de desarrollo. La IA fue empleada principalmente para:

- Corrección y optimización de scripts: Revisión del código en Python para asegurar el cumplimiento de las complejidades teóricas, el correcto manejo de estructuras y las buenas prácticas de programación.
- Mejora y refinamiento de redacciones: Corrección de estilo, ortografía y adecuación de la estructura formal del informe a la plantilla oficial de la cátedra.
- Explicación y depuración de errores: Diagnóstico detallado de mensajes de error y `tracebacks` (como `AttributeError`, `ModuleNotFoundError`, etc.) surgidos durante la ejecución de los algoritmos y pruebas unitarias.

Todo el código, las pruebas empíricas y los análisis presentados en este informe fueron validados, probados y respaldados bajo la estricta supervisión y responsabilidad del equipo de estudiantes.

Referencias y Bibliografía

1. Bradley N. Miller, David L. Ranum. Solución de problemas con algoritmos y estructuras de datos usando Python. Luther College, 2018.
2. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms. 4th Edition, MIT Press, 2022.
3. Apuntes de Cátedra. Unidad 2: Diseño y Análisis de Algoritmos — Clases de Complejidad. Facultad de Ingeniería, UNER, 2026.